

Solid Principles

ايه هي ال Solid والغرض منها ايه ؟!

هي 5 مبادئ أساسية هدفها تحسين جودة الـ **Software** . الغرض منها إن الكود يكون **Clean** ، والأهم من كده إنه يكون **Maintainable** (سهل الصيانة) و **Scalable** (سهل التوسع والتطوير) ومع الوقت بقي بيتم تطبيقهم في السوفتوير كله .
غرضها الأساسي انه يخلي السوفتوير اللي انت بتبنيه احسن وأقوى ، بصيغة ثانية يكون **Solid** اكثر

ولو سألك: يعني إيه الكود يبقى **SOLID** ؟ "رد بكلمة واحدة" يعني كود **Decoupled** بمعنى إن أجزاء الكود مش "ملزقة" في بعضها بطريقة تخلي التغيير مستحيل ، لا.. هي أجزاء مستقلة بتكلم بعضها بذكاء .

تعالى بقا نتكلم عن الخمس مبادئ

(1) الـ **Single Responsibility (S)**

ايه هو ال **Single Responsibility**

- الـ **SRP** له اكثر من تعريف او شرح ، لكن كلهم بينصبوا حول نفس الفكرة بالظبط ، وهي إن الحاجة تعمل شيء واحد فقط وبالتالي السبب لتغييرها هو سبب واحد فقط .
- ← **SRP** معناها ان كل **Function** او **Class** او **Model** له مسئولية واحدة فقط يقوم بيها وله سبب واحد فقط في التغيير
- ◆ المبدأ دا كل ال عايزه منك انك تخلي كل **Class** مسئول عن وظيفة واحدة بس .. وبكدا يكون فيه سبب واحد ممكن تغير او تعدل ال **Class** عشانه

تعالى نوضح اكثر بمثال

← لو فيه **class** مسئول انه يضيف **Employee** ف ال **database** وبعدين بيعتله **email** انه تم اضافته
-ف هنا ال **Class** المفروض يكون مسئول عن حاجة واحدة بس وهي انه يضيف ال **Employee**

❌ لكن انه بيعتله ايميل فدي كدا حاجة تانية

✓ الصح (تطبيق الـ SRP) إننا نخلي الكلاس مسئول عن "وظيفة واحدة" بس وهي عملية الإضافة، ونقسم المهام التقنية على 2 Interfaces

1) الـ Interface الأول (IRepository) : ده (Interface) مسئول عن أي حاجة تخص الداتايز.

2) الـ Interface الثاني (IService) : ده Interface مسئول عن أي حاجة تخص الإيميلات.

✓ إزاي بنربطهم؟ الكلاس المسئول عن الإضافة بينادي على ميثود Save من الـ Repository وميثود SendEmail من الـ IService من خلال الـ Interfaces دي .

✓ كدا انت خليت كل Class مسئول عن وظيفة واحدة بس .. واحد مسئول انه يضيف والثاني مسئول انه بيعت الايميل

طب استفدت كدا اي لما طبقت المبدأ دا ؟؟

◆ كدا انت قسمت Proget لأجزاء صغيرة وبالتالي لو حصل اي مشكلة هتوصلها بسهولة عشان تحلها

◆ كمان كل Method لما تبقي مسئولة عن حاجة واحدة بس فانت تقدر تعيد استخدامها ف اكتر من مكان

◆ دا غير انه هيققل اعتماد الـ Classes ع بعض بالتالي لما اعدل ف مكان مش هلاقي مكان ثاني وقع ويحصل اخطاء:

❌ الكود قبل التطبيق (غلط - مخالف لـ SRP)

هنا الكلاس بيعمل كل حاجة، وده بيخليه "ثقيل" وصعب في التعديل.

```

public class EmployeeService
{
    public void AddEmployee(Employee emp, String Email)
    {
        // كود إضافة الموظف للبيانات 1.
        Console.WriteLine("Saving Employee to Database...");

        // كود إرسال الإيميل (محتور جوه نفس الكلاس) 2.
        Console.WriteLine("Connecting to Email Server...");
        Console.WriteLine($"Sending welcome email to {emp.Email}...");
    }
}

```

✓ الكود بعد التطبيق (صح - يطبق SRP)

الشرح: الـ Interfaces دي بتضمن لي إن أي كلاس هيشغل "مخزن" لازم يكون فيه ميثود اسمها `SaveToDatabase..` , `SendEmail` مهما كان نوع الداتا بيز.

```

// Interfaces مسنول عن عمليات قاعدة البيانات فقط
public interface IEmployeeRepository {
    void SaveToDatabase(string name);
}

// Interfaces مسنول عن عمليات المراسلات فقط
public interface IEmailService {
    void SendEmail(string emailAddress, string message);
}

```

الشرح: كل كلاس هنا له مسؤولية واحدة فقط (SRP) الـ **SqlRepository** ملوش دعوة بالإيميلات، والـ **GmailService** ميعرفش حاجة عن الداتابيز.

```
// SQL Server كلاس متخصص في التعامل مع
public class SqlRepository : IEmployeeRepository {

    public void SaveToDatabase(string name) {
        // الفطلي SQL هنا كود الـ
        Console.WriteLine($"[DB]: تم حفظ {name} في SQL Server.");
    }
}

// Gmail كلاس متخصص في إرسال الإيميلات عبر
public class GmailService : IEmailService {

    public void SendEmail(string email, string message) {
        // Gmail الخاص بـ SMTP هنا كود الـ
        Console.WriteLine($"[Email]: تم إرسال '{message}' إلى {email} عبر Gmail.");
    }
}
```

الكلاس ده هو اللي بيحقق مبدأ الـ **Single Responsibility** في "إدارة العملية". هو مبيعرفش "إزاي" الموظف اتسيف، هو بس بيأمر الـ **Repository** إنه يسيف، وبعدين بيأمر الـ **EmailService** إنه يبعث الرسالة

```
public class EmployeeManager {

    private readonly IEmployeeRepository _repo;
    private readonly IEmailService _emailService;

    // Dependency Injection: المدير بيستلم الأدوات كـ
    public EmployeeManager(IEmployeeRepository repo, IEmailService emailService) {
        _repo = repo;
        _emailService = emailService;
    }

    public void RegisterNewEmployee(string name, string email) {
        // المهمة الأولى: الحفظ
        _repo.SaveToDatabase(name);

        // المهمة الثانية: الترحيب
        _emailService.SendEmail(email, "أهلاً بك في الفريق");
    }
}
```

رابعاً: التشغيل (Main Program)

هنا بنشوف القوة الحقيقية للفصل ده.

```
static void Main() {  
    // بنحدد إنا هستخدم أي أنوات النهاردة  
    IEmployeeRepository database = new SqlRepository();  
    IEmailService email = new GmailService();  
  
    // بنركبهم في المدير  
    EmployeeManager manager = new EmployeeManager(database, email);  
  
    // تنفيذ العملية  
    manager.RegisterNewEmployee("محمد صبحي", "m.sobhy@email.com");  
}
```

2) الـ Open-Closed Principle (OCP)

أي هو (OCP) Open-Closed Principle ؟ 😊 معناه ان بيسمح بالاضافه وغير مسموح بالتعديل & الكود بتاعك لازم يكون Open عشان تضيف فيه Methods جديدة لكن Closed بحيث متغيرش في الجزء اللي شغال كويس

ناخد مثال

--لو عندك Class اسمه NotificationService ودا من اسمه مسئول انه بيعت Notification ف هيكون فيه Attributes زي ال Message ودي هتكون String وفيها الرسالة ال هتتبع وكمان ال type عشان نعرف نوع Notification ال هتتبع وفانكشن Send مسئولة انها تبعت ال Notification بس الاول بنشوف النوع لو هتتبع ع ال Email ولا مثلاً Sms او غيره ودا هيتعمل ب if .. else if وكدا

✗ يعني كدا كل ما تحتاج تضيف نوع Notification جديد فأنت هتضطر تروح ع ال Class وتعديل ف ال function بتاعته بأنك تزود if else جديدة وهكذا ودا مش صح

فيركز على جعل الكود مفتوحاً للتوسع ومغلقاً للتعديل OCP

Open > Extended مسموح بالتوسع والاضافه

Close > Modified غير مسموح بالتعديل علي الكود الحالي

✗ الكود قبل تطبيق الـ OCP (مخالف للمبدأ)

هنا الكلاس "مفتوح للتعديل (Open for Modification)" ، وده اللي بنحاول نتجنبه.

```
public class NotificationService
{
    public void Send(string type, string message)
    {
        if (type == "Email")
        {
            Console.WriteLine("Sending Email: " + message);
        }
        else if (type == "SMS")
        {
            Console.WriteLine("Sending SMS: " + message);
        }
        else if (type == "WhatsApp")
        {
            Console.WriteLine("Sending WhatsApp: " + message);
        }
        // وهاكذا ...
    }
}
```

--طب هنحل المشكلة هنا ازاي؟؟

✓ المشكلة هنا هنتحل لو كنت مطبق الـ OCP من الاول - 😊

طب اي هو دا وازاي اطبقه ؟

المبدأ دا بيقولك ان اي Class او فانكشن عندك المفروض انها تكون مقفولة عن اي تعديلات .. ولكن بيبقى مفتوح عادي لأي اضافات جديدة

المبدأ دا بيطبق عن طريق الـ Abstract Class او الـ Interface تعالي نشوف ازاي بأننا نحل المشكلة الـ قابلتنا فوق

✓ هنخلي ال Notification كلاس يكون .. Abstract بأن ال Send فانكشن مش هيبقي جواها اي كود وكمان هنشيل ال type attribute لأننا مش هحتاجها ونعمل كلاس جديد وليكن EmailNotifier ونخليه يورث من كلاس ال Notification وبكدا هو لازم يكتب الكود بتاعت ال Send فانكشن ع حسب بقي هو هيبعت ازاي وكلاس كمان SMSNotifier وبرضو هيورث من كلاس ال Notification ويكتب الكود بتاع ال Send فانكشن زي ما هو عايز

✓ يبقي كدا كل ما احتاج اضيف نوع جديد انا مش هعدل ع اي كود كان موجود قبل كدا لا هعمل بس Class جديد للنوع دا واخلية يورث من Class ال -- Notification طب المبدأ دا كدا فادني ب ايه؟؟ ✨ المبدأ دا من اهم المبادئ ال هتخليك تضيف Features جديدة من غير ما تعدل ع الكود القديم لأنك لو عدلت ممكن تحصل مشاكل ف اماكن تانية دا غير انه هو كمان هيدعم المبدأ ال فات وهو ال Single Responsibility

الكلاس ده Closed for Modification مش هنعديل فيه تاني، هو مجرد "قالب ال Abstraction إحنا هنا بنقول: "أي كلاس عايز يبعث رسائل لازم يكون عنده ميثود اسمها Send لا الكلاسات دى تعرف إزاي الرسالة بنتبعث، ولا يهملها النوع.

```
public abstract Notification {
    abstract void Send(string message);
}
```

✓ لاحظ هنا إن البرنامج يتعامل مع النوع العام (**Notification**) وميفرقش معاه هو إيه النوع الفعلي اللي هيتبعته، هو بينادي على **Send** وبس.

```
public class EmailNotifier : Notification
{
    public override void Send(string message)
    {
        Console.WriteLine($"[Email]: {message}");
    }
}

public class SmsNotifier : Notification
{
    public override void Send(string message)
    {
        Console.WriteLine($"[SMS]: {message}");
    }
}
```

✓ ده أجمل حته في الـ **OCP** الكلاس ده "مغلق" تماماً لو ضفت 100 نوع إشعار جديد، الميثود دي مش هتتغير ولا سطر، لأنها بتتعامل مع الأب (**Notification**) مش مع الأنواع الفرعية.

```
public class NotificationService
{
    public void Execute(Notification notification, string msg)
    {
        // بمرر الرسالة وقت التنفيذ
        notification.Send(msg);
    }
}
```


(5) التجربة العملية (Entry Point)

اسم الملف Program.cs :

```
static void Main(string[] args)
{
    NotificationService service = new NotificationService();
    Notification email = new EmailNotifier();
    service.Excuet(email,"welcome to");
    Notification Sms = new SmsNotifier();
    service.Excuet(Sms,"code is 463");
}
```

Liskov Substitution (LSP) (3)

مبدأ **Liskov Substitution** : بيقول يجب ان تكون **Childs Classes** قادرة على ع ان
تحل محل **Pernt Classes** دون ان تؤدي الى كسر او تغيير في سلوك ال **Code**

يعني لو عندك **Perent Class** وورثت منه **Child class** يبقى لازم تقدر تستخدم **Child Class** مكان **Perent** من غير ما يحصل مشاكل او تغيير ف سلوك البرنامج

يعني ال **Child** لازم يشتغل زي من **Perent** من غير مفاجات

← طب مثال: تخيل عندك عربية كبيرة وعربية صغيرة. الاتنين دول عربيات، يعني ليهم نفس ال **Properties** (عجل، موتور، فرامل). لو مكان ما بيقبل بس العربيات الكبيرة، تقدر تدخل العربية الصغيرة بدالها عادي، لأنها برضه عربية

الكود قبل تطبيق الـ LSP (مخالف للمبدأ)

مثال في الـ Code ::

* الغلط هنا استخدمت Soap مكان juice لانهم نفس العيلة لكن سلوك الـ Soap مش زي الـ Parent بيكسر البرنامج ودا كذا كسر مبدأ Liskov Substitution

مثال فيه مشكله بيكسر مبدأ LSP

```
public class juice
{
    public virtual void drink()
    {
        Console.WriteLine("you drink juice");
    }
}

public class orangejuice : juice
{
    public override void drink()
    {
        Console.WriteLine("you drink juice");
    }
}

class soap:juice    // المشكله هنا
{
    public override void drink() // لان ده صابون ي مجنون مش بيتشرب
    {
        throw new Exception("ostrich cant fly");
    }
}

private static void Main(string[] args)
{
    juice b = new soap(); // استبدلنا الاب بالابن
    b.drink(); // البرنامج ضرب لان الصابون مش بيتشرب
}
```

اللى بيتشرب طبق مبدء IDrinkable واللى مشبيتشرب زى Soap ملهوش علاقة

```
1: interface IDrinkable { void Drink(); }

2: class juice : IDrinkable
{
    public void Drink()
    {
        Console.WriteLine("you drink juice");
    }
}

3: class orangejuice : juice
{
    public void Drink()
    {
        Console.WriteLine("you drink juice");
    }
}

4: class soap
{
    public void wash() // لان ده صابون ي مجنون مش بيتشرب
    {
        throw new Exception("you wash your hands withe soap ");
    }
}

private static void Main(string[] args)
{
    IDrinkable i = new juice();
    i.Drink();
    IDrinkable ii=new orangejuice();
    ii.Drink();

    IDrinkable soap = new soap(); // الصابون مش بيتشرب فمش اقدر اكتبه كدا
}
```

Interface Segregation (ISP) (4)

"بينص على الكلاسات ماينفعش تُجبر على تنفيذ (methods) مش بتحتاجها".

بمعنى، بدل ما أعمل Interface كبير فيه methods كتير غير مرتبطة ببعض، بقسمه لـ Interfaces صغيرة ومتخصصة، وكل Class تطبق بس اللي هي محتاجاه.

ده بيخلي الكود أبسط، أسهل في الفهم، وبيققل التغيير غير الضروري لما يحصل تعديل أو إضافة.

مثال للتوضيح

تصور نفسك بتعمل أكل، عندك طبّاخ بيعمل الرز، وطباخ تاني بيعمل اللحم، وطباخ ثالث بيعمل السلطة. كل واحد متخصص في حاجة معينة. لو كل طبّاخ لازم

يعمل كل الأكل، هيبقى الأكل مش مضبوط وهياخد وقت كثير

بساطة، مبدأ **Interface Segregation** بيقول:

ما تخليش العميل يتعامل مع حاجات مش محتاجها. خلي كل **Interface** مسئولة عن جزء محدد من الشغل.

ليه ده مهم؟

بيقل التعقيد: كل **Interface** بتكون مسئولة عن جزء بسيط، فبيسهل فهم الكود وتطويره.

• بيزيد المرونة: لو عايز تضيف **method** جديدة، هتضيفها في **Interface**

المناسبة من غير ما تغير في باقي النظام.

بيحسن الصيانة: لو فيه مشكلة في جزء معين، هتقدر تصلحها بسرعة من غير ما تؤثر ع باقي الاجزاء

مثال :: المشكلة **Class User**: مضطر يطبق دوال مش محتاجها (**sendEmail, log**) وده ضد مبدأ **Interface Segregation**.

```
interface IUser {
    void login();
    void register();
    void sendEmail();
    void log();
}

class User implements IUser {
    public void login() {
        // login logic
    }

    public void register() {
        // register logic
    }

    public void sendEmail() {
        // User محتاج الدالة دي
    }

    public void log() {
        // User محتاج الدالة دي
    }
}
```

✓ الكود بعد إعادة التنظيم والتقسيم الصح:

✖ الخطوة 1: تقسيم الـ Interfaces

```
// مسؤولة عن إدارة المستخدم
interface IUser {
    void login();
    void register();
}

// Interface لإرسال الإيميلات
interface IEmail {
    void sendEmail();
}

// Interface لتسجيل الأحداث أو الأخطاء
interface ILogger {
    void log();
}
```

✖ الخطوة 2: تطبيق كل Class للـ Interface اللي بيحتاجها فقط

```
// Class عن المستخدم فقط
class User : IUser {
    public void login() {
        // login logic
    }

    public void register() {
        // register logic
    }
}
```

```
// Class مسؤول عن إرسال الإيميلات فقط
class EmailService : IEmail {
    public void sendEmail() {
        // send email logic
    }
}
```

```
// Class مسؤول عن اللوج فقط
class LoggerService : ILogger {
    public void log() {
        // log logic
    }
}
```

ملاحظات: 

- كل كلاس يطبق بس الـ **Interface** اللي بيحتاجها.
- لو حصل تغيير في **IEmail** مش هياثر على **User** لأن مفيش بينهم علاقة مباشرة.
- ده بيخلي الكود مرن، نظيف، قابل للتوسعة والصيانة بسهولة.

Dependency inversion (5)

المبدأ ده بيقول ببساطة:

المبدأ بيقول حاجتين:

1. الكلاسات الكبيرة (**High-level**) ما تعتمدش على الكلاسات الصغيرة (**Low-level**).

2. الاتنين لازم يعتمدوا على **Abstract** (يعني **Interface** أو **Abstract Class**).

✓ ما تخلّش كلاس يعتمد على كلاس تاني بشكل مباشر، خليه يعتمد على واجهة (**interface**) بدلاً من كده.

👁️ ليه؟ علشان لو الكلاس اللي تحت اتغير، الكلاس اللي فوق مايتأثرش!

📦 تخيل معايا المثال ده:

🌀 كود غلط مافيهوش: (**Dependency Inversion**)

```

public class ClassA
{
    public void MethodA()
    {
        Console.WriteLine("أنا Class A");
    }
}

public class ClassB
{
    ClassA a;

    public void MethodB()
    {
        a = new ClassA();    // ← ClassB يخلق ClassA
        a.MethodA();         // ← يعتمد عليه تمامًا!
    }
}

```

في هذا الكود، **ClassB** يقوم بإنشاء **object** من **ClassA** داخل **MethodB**، مما يعني أن **ClassB** يعتمد بشكل مباشر على **ClassA**

& المشكلة:

ClassB يعتمد على **ClassA** بشكل مباشر، وهذا يُعد انتهاكًا لمبدأ عكس **Dependency** (**Inversion** يعني "مرتبط بيه ارتباط مباشر"، ومينفعش **ClassB** يشتغل من غيره بل يجب أن يعتمد على (**interface** أو **abstract**)

طبيب إيه المشكلة؟ ليه ده غلط؟ 

المشكلة إنك كده قيدت **ClassB**، وبقيت تقول:

"أنا مش هعرف أشتغل إلا بـ **ClassA**، ومينفعش أشتغل بحد غيره".

فالو:

• **ClassA** اتغير أو اتحذف أو اتبديل بكلاس تاني

✗ لازم تغير **ClassB** كمان

وده بيخلي الكود مربوط ببعضه جدًا، وده اسمه **Tightly Coupled**

طيب نعمل إيه؟

نستخدم Dependency Inversion Principle

يعني نخلي ClassB يقول:

"أنا مش هتعامل مع ClassA نفسه، أنا هتعامل مع حاجة " (interface) "اسمها...IClassA ومش فارق مين اللي يحققها ClassA أو غيره".

كود صح فيه: (Dependency Inversion)

1. نعمل Interface

```
Interface نعمل
public interface IClassA
{
    void MethodA();
}

نخلي ClassA يطبق Interface
public class ClassA : IClassA
{
    public void MethodA()
    {
        Console.WriteLine("لنا Class A");
    }
}
```

3. نخلي ClassB يعتمد على (interface) بدل من الكلاس:

```
public class ClassB
{
    IClassA a;

    // Constructor Injection هستخدم
    public ClassB(IClassA a)
    {
        this.a = a;
    }

    public void MethodB()
    {
        a.MethodA(); // بيشغلها من غير ما يعرف هي أنهي كلاس بالضبط
    }
}
```


ونشغل الكود كده:

```
class Program
{
    static void Main(string[] args)
    {
        IClassA a = new ClassA(); // أو أي حاجة ثانية ClassA2 ممكن في المستقبل نستخدم
        ClassB b = new ClassB(a); // نبعتله الكلاس من برة
        b.MethodB(); // MethodA هينفذ اللي جوه
    }
}
```

ClassB مش بيعرف حاجة عن ClassA

الآن ClassB يعتمد على التجريد (Interface) وليس على ClassA بشكل مباشر.

يعني بيقول : أنا مش فارق معايا مين اللي ينفذ، المهم يكون فيه Method اسمها " (MethodA

💡 جملة تحفظها للإنترفيو:

classB "يعتمد على abstraction مش implementation ، وده ببسهل التعديل، والاستبدال، واختبار الكود، وبيخلي النظام". loosely coupled

عمر ك سالت نفسك ليه ف العموم ليه إنا بنفضل التعامل مع الـ Interface مش الكلاسات
التعامل مع الكلاسات مباشرة بيخلي الكود Tightly Coupled زي المكعبات الملحومة في بعض بـ
غراء. لو حاولت تفك قطعة، القطعة اللي فوقها هتتكسر. التعامل مع الـ Interface بيخلي الكود
Loosely Coupled زي مكعبات الـ Lego تقدر تشيل قطعة وتركب غيرها في ثانية طالما القطعة
الجديدة ليها نفس البروز. (Interface)

اللهم انفعني بما علمتني، وعلمني ما ينفعني، وزدني علماً. اللهم إني أسألك فهماً في " العلم، وبركة في العمل، وإخلاصاً في النية. اللهم اجعل ما تعلمته وسيلة لنفع الناس وتيسير حياتهم، وافتح لي أبواب فضلك وكرمك يا رب العالمين.